- Cannot handle high dimensional data efficiently
- High Time Complexity

****

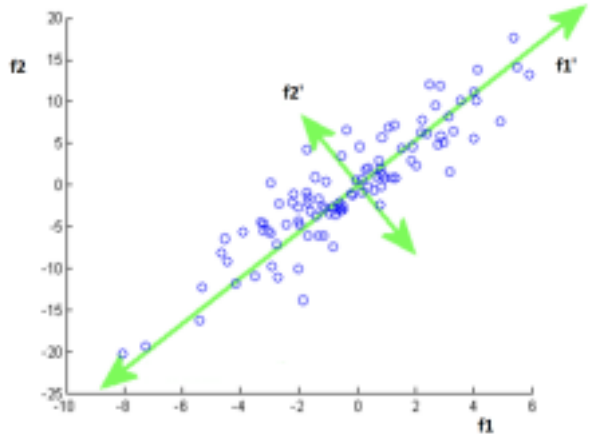
High Dimensional Visualisation

1. Principal Component Analysis (PCA)

- Dimensionality reduction techniques help to convert high dimensional data to fewer dimensions which can be then visualized using simpler plots or it can be used when we want to preserve the information of our feature columns.

- Principal component analysis, or PCA, is a dimensionality-reduction method that is often used to reduce the dimensionality of large data sets.
- PCA is an act of finding a new axis to represent the data so that a few principal components may contain the most information.
- The main objective here is whenever we are going from a higher dimension(d) to a lower dimension(d') we want to preserve those dimensions which have high variance (or high / information.)

For example: Imagine we have two features f1 and f2, and the data is spread as shown in the image below. We want to project the given 2-D data to 1-D.



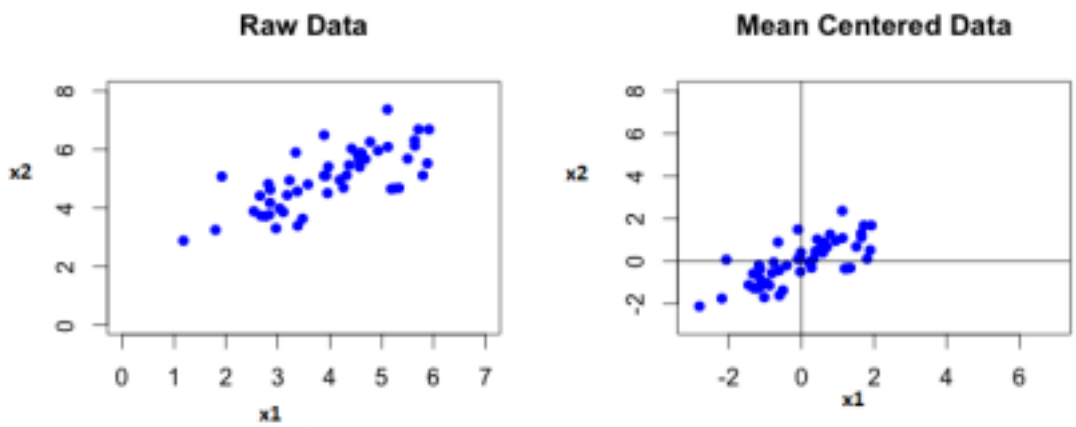
- One thing we can observe here is that both axes have a good amount of variance. It will be difficult to decide which feature to drop.

- Therefore, we rotated the entire coordinate axes. It is shown in green.

- Let's call these new dimensions as $\mathbf{f_1}'$ and $\mathbf{f_2}'$. Now we can see that variability on $\mathbf{f_1}'$ is more than $\mathbf{f_2}'$ and we can now drop the $\mathbf{f_2}'$ axis.

Mathematics of PCA:

Step 1: Feature wise Centering: We compute the mean of each feature and subtract it from each value of that feature to center the mean of the data at origin.



Step 2: Standardize: It is done in order to keep all the features of our data in same scale. Best is to use standard scaling because it is less affected by the outliers.

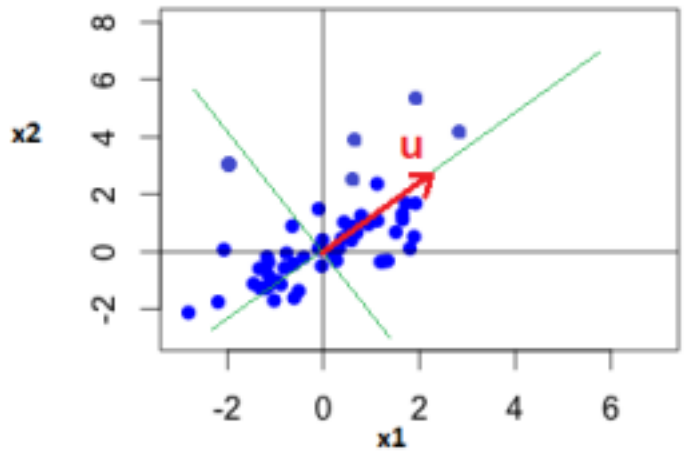
Combining step1 and step 2, we get the value of our feature vector x as:

$$x' = \frac{x - x.mean()}{x.std()}$$

Step 3: Rotate the coordinate axes:

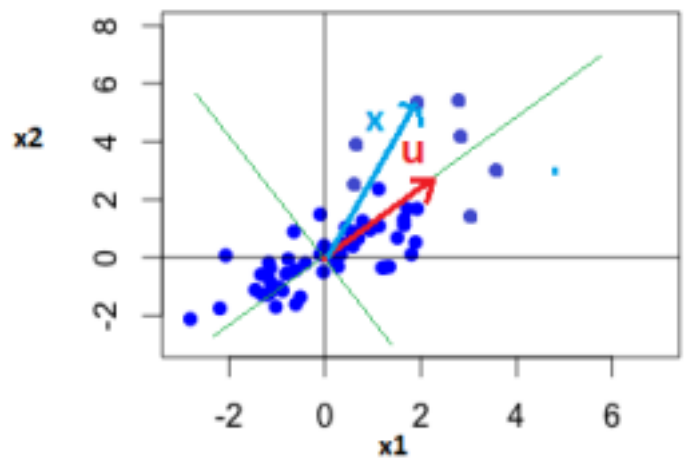
- There can be many angles of rotation possible, we'll consider a random axis and optimize it to get the best axis.
- Let's consider a unit vector in the direction of the new axis. $\vec{u}$

- Our optimization problem is finding unit vectors u such that the variance of all $\mathbf{x}_i'$ (features) are maximum.



- We can also think of it in terms of the length of projections.

- Consider a vector x in the space representing one of the points in our data.



- The projection of a vector x on vector u is given as: $\frac{\vec{u}.\vec{x}}{||\vec{u}||}$

- The best u will be where the summation of the length of projections of all such points(xi) on the vector u is maximum.

i.e. $$\max_{\vec{u}} \frac{1}{n} \sum_{i=1}^{n} \frac{\vec{u}.\vec{x_i}}{||\vec{u}||}$$

- However, the projections may be a negative value. Therefore, we take the magnitude of the numerator:

i.e. $$\max_{\vec{u}} \frac{1}{n} \sum_{i=1}^{n} \frac{|\vec{u}.\vec{x_i}|}{||\vec{u}||}$$

- Since the above objective function is not differentiable, we rewrite it as:

$$\max_{\vec{u}} \frac{1}{n} \sum_{i=1}^{n} \frac{(\vec{u}.\vec{x_i})^2}{||\vec{u}||}$$

- Now, since u is a unit vector, we introduce a constraint: $||\mathbf{u}|| = 1$

- Using Lagrange's multiplier, our loss function becomes:

$$max_{\vec{u},\lambda} \quad \frac{(X.\vec{u})^2}{n} + \lambda(||\vec{u}|| - 1)$$

We can optimize the above Lagrangian loss function using Gradient descent.

Optimization without Gradient descent:

- We know that, $\vec{u}.\vec{x_i} = \vec{x_i}.\vec{u}$

therefore, our new loss function is:

$$L = \frac{(X.\vec{u})^2}{n} + \lambda(||\vec{u}|| - 1)$$

- Now, since we can write the constraint $||\vec{u}|| = 1$ as $||\vec{u}||^2 = 1$, therefore,

$$L = \frac{(X.\vec{u})^2}{n} + \lambda(||\vec{u}||^2 - 1)$$

- Now we know that If a matrix A is an invertible square matrix, then:

$$A^2 = A^T.A = ||A||^2$$

Thus,

$$L = \frac{(X.\vec{u})^T(X.\vec{u})}{n} + \lambda((u^T.u) - 1)$$

- Using the identity, $(A.B)^T = B^T.A^T$

We get,

$$L = \frac{(u^T.X^T)(X.\vec{u})}{n} + \lambda((u^T.u) - 1)$$

Or $$L = u^T \frac{X^T X}{n} u + \lambda(u^T u - 1)$$

Let $\frac{X^T X}{n} = V$

- Hence, $L = u^T V u + \lambda(u^T u - 1)$

V is the pairwise covariance matrix of all the features of our feature matrix X.

- Taking partial derivatives w.r.t μ and λ,

$$\frac{\partial L}{\partial u} = 2Vu + 2\lambda u \quad \text{and} \quad \frac{\partial L}{\partial \lambda} = u^T u - 1$$

- After putting both the partial derivatives equal to zero, we get,

$$Vu = \lambda u \quad \text{— (i)}$$

$$u^T u = 1$$

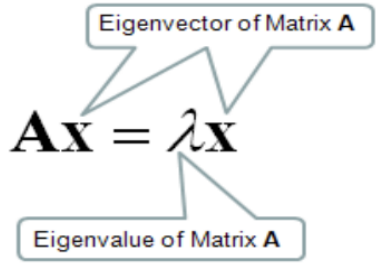
- By carefully observing the equation (i), we can conclude that u is the eigenvector and λ is the eigenvalue of matrix **V**.

Eigenvector and Eigenvalue:

- For any matrix A, there exists a vector **x** such that when this vector is multiplied with the matrix A, we get a new vector in the same direction having a different magnitude.

  - The vector **x** is called the Eigenvector and the length is called as

Eigenvalue. i.e.



- There can be multiple eigenvectors, which are always orthogonal to

each other.

Conclusion:

- To find the best value of vector $\mathbf{u}$, which is the direction of maximum variance (or maximum information) and along which we should rotate our existing coordinates, we follow the below-given steps:

  Step 1: Find the covariance matrix of the given feature matrix X.

  Step 2: Then calculate the eigenvectors and eigenvalues of the covariance matrix. The eigenvector is the direction of best $\mathbf{u}$ and the eigenvalue is the importance of that vector.

- The eigenvector associated with the largest eigenvalue indicates the direction in which the data has the most variance.

- Therefore, we can select our principal components in the direction of the eigenvectors having large eigenvalues and drop the principal components having relatively small eigenvalues.

2. t-SNE (t-Distributed Stochastic Neighbor Embedding)

- The core idea of t-SNE is to preserve the pairwise distance in a neighborhood when the datapoint in a higher dimensional space is projected into a lower dimensional space.

- Therefore, t-SNE tries to create an embedding that preserves the neighborhood using some probabilistic methods.

Ideas for the probabilistic model

- We need to define the probability that a point x_i is a neighbor of x_j.

- One way is to define probability as inversely proportional to the distance between points.

i.e.

$$p_{ij} = \frac{1}{||x_i - x_j||}$$

- But there are some problems with the above probability function:
    - The function gives an infinite value when the distance between the two points is zero.
    - Probability reduces too fast.
    - The function depends on an absolute function.

- Therefore, we take inspiration from Gaussian distributions for a better PDF and a new probability function is defined as:

$$p_{ij} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq l} \exp(-\|x_k - x_l\|^2 / 2\sigma_i^2)}$$

- The standard deviation with respect to each x_i creates a scaled distance effect in the neighborhood of x_i and the denominator ensures that the sum of probabilities is 1.

Math for t-SNE

- Our objective is to project data points $x_i \in R^d$ to y_i using t-SNE, where $y_i \in R^2$
- We compute the pairwise similarities as probabilities.
- We compute P_{ij} for d-dimensions and Q_{ij} for d'-dimensions where d>d.

i.e.

$$p_{ij} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq l} \exp(-\|x_k - x_l\|^2 / 2\sigma_i^2)}$$

where, $\mathbf{p_{ij}}$ is the probability that the points x_i and x_j are neighbors in *d*-dimensional space, and $\mathbf{q_{ij}}$ is the probability that the points x_i and x_j are neighbors in *d'*-dimensional space.

- We define $\mathbf{q_{ij}}$ with the same formulation as $\mathbf{p_{ij}}$ since every x_i and x_j would have corresponding y_i and y_j in d' dimensional space.

$$q_{ij} = \frac{exp(-||y_i - y_j||^2)}{\sum_{k \neq l} exp(-||y_k - y_l||^2)}$$

- Now, to get a useful d' transformation we need $p_{ij} \approx q_{ij}$ **.** Thus, we need a loss function to optimize.

KL divergence

- It is a loss function that measures the dissimilarity between two distributions.
- The KL-Divergence between two distributions P and Q is written as:

$$KL - div(P_{ij}, Q_{ij}) = \sum_i \sum_j [P_{ij} \ . \ log(\frac{P_{ij}}{Q_{ij}})]$$

- Some interpretations using the KL-divergence formula are:
  - If P_{ij} and Q_{ij} are the same, then KL-divergence will be equal to 0.
  - If Pij is very small and P_{ij} and Q_{ij} are different, then KL-divergence will have a small value.
  - Lastly, since KL-divergence is a measure of dissimilarity, it is always **greater than or equal to 0.**

    - Here, P is working as a weightage because if P_{ij} is small we don't really care as points x_i and x_j will be far away from each other in d-dimension space.

- Thus, our **optimization problem** would be to find all the y_is that **minimize** KL-divergence(P,Q).

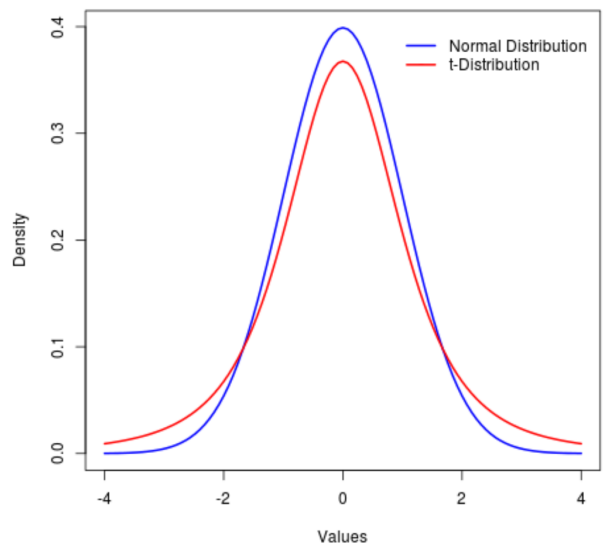
T-distribution (Cauchy Distribution)

- Researchers proposed the use of t-distribution with a degree of freedom equal to 1 instead of gaussian distribution for dimension d'.

- New q_{ij} is defined as:

$$q_{ij} = \frac{(1 + ||y_i - y_j||^2)^{-1}}{\sum_{k \neq l}(1 + ||y_k - y_l||^2)^{-1}}$$

- t-distributions work better than gaussian distributions with SNE because:
    - t-distributions with a degree of freedom equal to 1 have a longer tail than Gaussian distributions
    - Gaussian distribution falls exponentially while t-distributions sort of inversely
    - So, for our q_{ij}s , if we use t-distribution, two points can go farther away and still get pairwise distance preserved of the sort. Therefore, it is able to capture distant neighbors better.



- Therefore, we use Gaussian distribution for p_{ij}s and t-distribution for q_{ij}s because of which if two points are far away in lower dimensional space, the probabilities will still remain the same.

Perplexity

- Perplexity can be interpreted as the effective number of neighbors whose distance we want to preserve
- It is one of the parameters that we might want to configure when using t-SNE.
- Its value is kept in the range [5, 50]

****